

In your **Credit Card Fraud Detection** dataset, **V1 to V28** are **anonymized numerical features**. They represent information extracted from the original transaction data, but their exact real-world meanings are not provided.

### **Why are they called V1, V2, ..., V28?**

The dataset was anonymized to protect sensitive financial information. Instead of showing original details such as merchant, location, or transaction type, the dataset provides transformed numerical features.

For example:

| Column | What it means                |
|--------|------------------------------|
| V1     | Anonymized numerical feature |
| V2     | Anonymized numerical feature |
| V3     | Anonymized numerical feature |
| V4     | Anonymized numerical feature |
| ...    | ...                          |
| V28    | Anonymized numerical feature |

**Important:** We cannot say that V1 means "location" or V14 means "transaction amount." Their actual meanings are unknown.

### Example of your dataset

| Time | V1     | V2     | V3     | V4    | ... | Amount | Class |
|------|--------|--------|--------|-------|-----|--------|-------|
| 0    | -1.359 | -0.072 | 2.536  | 1.378 | ... | 149.62 | 0     |
| 406  | -2.312 | 1.952  | -1.609 | 3.997 | ... | 0.00   | 1     |

Here:

- V1 = -1.359 → A numerical value of anonymized feature 1
- V2 = -0.072 → A numerical value of anonymized feature 2
- V3 = 2.536 → A numerical value of anonymized feature 3
- Class = 0 → Genuine transaction
- Class = 1 → Fraudulent transaction

### What does the ML model do with V1–V28?

The model learns patterns from these features.

For example, **illustrative only**:

```
V1 = -2.31
V2 = 1.95
V3 = -1.60
V4 = 3.99
...
Amount = 0.00
```

The model may learn that certain combinations of feature values are associated with fraud.

It does **not** need to know what each feature means in the real world.

## Simple analogy

Imagine you have a student dataset:

```
Study Hours
Attendance
Previous Marks
Final Result
```

You can understand what each column means.

But in this fraud dataset, the original columns have been transformed into:

```
V1, V2, V3, ..., V28
```

So the model can still learn patterns, but **we cannot interpret each feature directly**.

## What should you say to your professor?

"V1 to V28 are anonymized numerical features obtained from the original credit card transaction data. Their exact semantic meanings are not disclosed, but they contain information useful for distinguishing fraudulent and genuine transactions. The model learns patterns from these features to predict fraud probability."

## One important correction to our earlier project plan

Because V1–V28 are anonymized, we should **not claim that the model is analyzing customer location, transaction frequency, or previous transaction history** unless those features are actually available in the dataset.

For this dataset, we should describe the project as:

**"An ML-based fraud detection system using anonymized transaction features."**

If you want a more realistic banking project later, we can create a **synthetic transaction dataset** containing features such as:

```
Customer ID
Transaction Amount
Transaction Time
Location
```

Merchant Category  
Previous Transaction Amount  
Number of Transactions in Last 24 Hours

Then we can build a more interpretable **risk-scoring system**.

**Example:**  $V_{14} = -4.2$  is a valid numerical input to the model, but we cannot honestly say, "The transaction is risky because  $V_{14}$  represents location."



how can we know whether transaction is fraud or not

We determine whether a transaction is **fraudulent or genuine** by using a **machine learning model that learns patterns from previously labeled transactions**.

In your project, the important column is `Class` .

## 1. First, understand the `Class` column

Your dataset contains transactions like:

| Time | V1     | V2     | Amount | Class |
|------|--------|--------|--------|-------|
| 0    | -1.359 | -0.072 | 149.62 | 0     |
| 406  | -2.312 | 1.952  | 0.00   | 1     |

Here:

Class = 0 → Genuine transaction  
Class = 1 → Fraudulent transaction

The `Class` column is the **answer that the model learns from during training**.

## 2. How does the model learn?

Suppose we give the model thousands of transactions:

Transaction A →  $V_1, V_2, \dots, V_{28}, \text{Amount}$  → Class 0  
Transaction B →  $V_1, V_2, \dots, V_{28}, \text{Amount}$  → Class 1  
Transaction C →  $V_1, V_2, \dots, V_{28}, \text{Amount}$  → Class 0

The model studies the relationship between:

Transaction Features → Fraud / Genuine

For example, it may learn that **certain combinations of anonymized feature values** are more common in fraudulent transactions.

**Important:** These are learned statistical patterns, not fixed rules such as “if  $V_{14} > 5$ , then fraud.”

### 3. What happens when a new transaction arrives?

Suppose a new transaction comes in:

```
Time = 1000
V1 = -1.2
V2 = 0.5
V3 = 1.1
...
Amount = 50000
```

The model receives these features:

```
New Transaction
    ↓
V1, V2, ..., V28, Amount
    ↓
Trained ML Model
    ↓
Fraud Probability
```

For example, the model might output:

```
Fraud Probability = 0.87
```

This means the model estimates a **high likelihood of fraud**.

### 4. How do we convert probability into a decision?

We use a **threshold**.

For example:

If probability  $\geq 0.50 \rightarrow$  Fraud  
If probability  $< 0.50 \rightarrow$  Genuine

## Example

| Fraud Probability | Decision |
|-------------------|----------|
| 0.02              | Genuine  |
| 0.15              | Genuine  |
| 0.49              | Genuine  |
| 0.87              | Fraud    |
| 0.99              | Fraud    |

So:

$$0.87 \geq 0.50$$

Therefore:

Predicted Class = 1  $\rightarrow$  Fraud

**Note:** The 0.50 threshold is only an example. In a real fraud detection system, we may choose a different threshold depending on how important it is to catch fraud versus avoid falsely blocking genuine transactions.

## 5. What is the actual ML formula?

For **Logistic Regression**, the model calculates a probability using:

$$P(\text{Fraud}) = \frac{1}{1 + e^{-z}}$$

where:

$$z = w_1 V_1 + w_2 V_2 + \dots + w_{28} V_{28} + w_{29} \text{Amount} + b$$

Here:

- $V_1, V_2, \dots, V_{28} \rightarrow$  Input features
- Amount  $\rightarrow$  Transaction amount
- $w_1, w_2, \dots, w_{29} \rightarrow$  Learned weights

- $b \rightarrow$  Bias
- $P(\text{Fraud}) \rightarrow$  Predicted fraud probability

## Example

Suppose the model calculates:

$$z = 1.9$$

Then:

$$P(\text{Fraud}) = \frac{1}{1 + e^{-1.9}} \approx 0.87$$

So the model predicts:

Fraud Probability = 87%

This is how Logistic Regression converts transaction features into a fraud probability.

## 6. But how do we know whether the model is actually correct?

This is where **model evaluation** comes in.

We already know the actual `Class` values for the test dataset.

For example:

| Actual Class | Model Prediction | Result       |
|--------------|------------------|--------------|
| 0            | 0                | Correct      |
| 1            | 1                | Correct      |
| 1            | 0                | Missed fraud |
| 0            | 1                | False alarm  |

These four cases are called:

- **True Negative (TN):** Genuine correctly identified
- **True Positive (TP):** Fraud correctly identified
- **False Negative (FN):** Fraud missed
- **False Positive (FP):** Genuine incorrectly flagged

## Example

Suppose we test 100 transactions:

```
Actual fraud transactions = 10
Model detects correctly   = 8
Model misses              = 2
```

Then:

$$Recall = \frac{TP}{TP + FN}$$
$$Recall = \frac{8}{8 + 2} = 0.80$$

So:

```
Recall = 80%
```

This means the model detected **80% of the actual fraud transactions**.

## 7. Very important: The model does not know the "true" fraud status of a new transaction

For a **new, unlabeled transaction**, the model can only say:

"Based on the patterns I learned, this transaction has a high probability of being fraudulent."

It cannot guarantee that the transaction is actually fraud.

For example:

```
Model Prediction:
Fraud Probability = 0.92
```

This does **not** mean:

"The transaction is definitely fraud."

It means:

"The transaction is highly suspicious according to the model."

In a real banking system, the transaction may then be:



High Risk  
↓  
Additional Verification  
↓  
Manual Review / Customer Confirmation  
↓  
Final Decision

## 8. How this fits into your project

Your complete fraud detection flow will be:

Historical Transactions  
↓  
Class = 0 / 1  
↓  
Train ML Model  
↓  
Learn Fraud Patterns  
↓  
New Transaction  
↓  
Predict Fraud Probability  
↓  
Apply Threshold  
↓  
Fraud / Genuine  
↓  
Calculate Risk Score

## Example final output

Transaction ID: TX1024

Fraud Probability: 0.87

Risk Score: 87/100

Risk Level: HIGH

Prediction: Fraud

## In one sentence for your professor:

"The system learns patterns from labeled historical transactions, uses the trained model to calculate the fraud probability of a new transaction, and applies a decision threshold to classify it as fraudulent or genuine."

Next, the most important thing to understand is: how does Random Forest or XGBoost actually learn these fraud patterns from V1–V28?



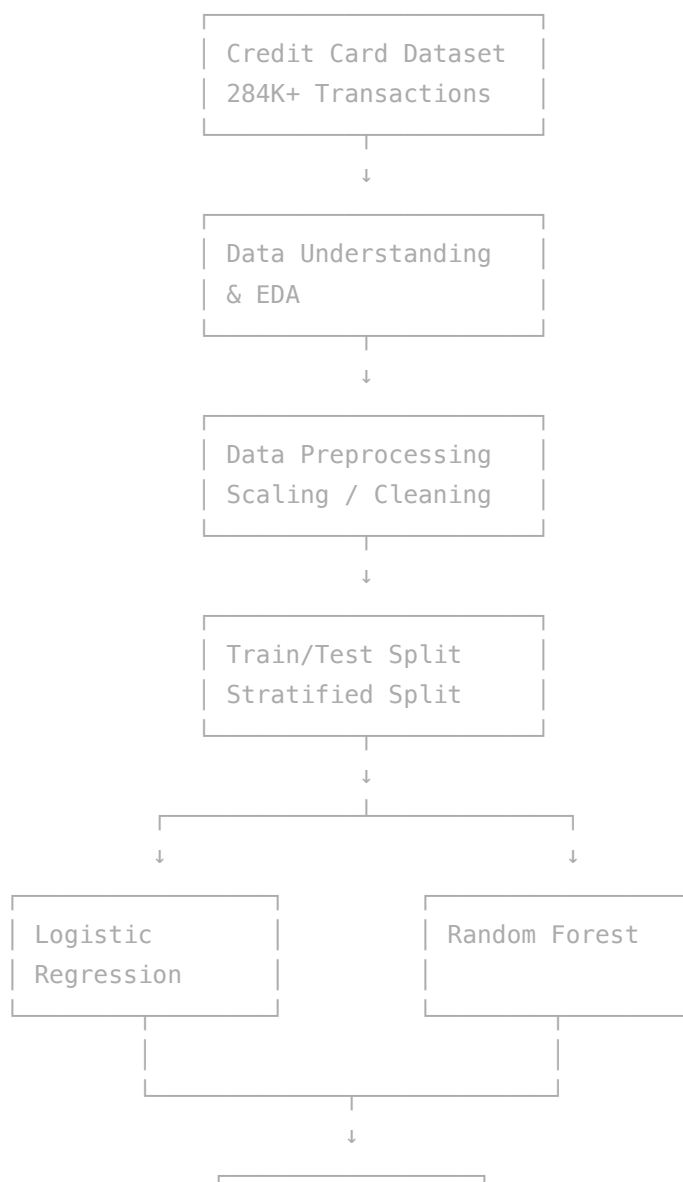
Today 9:59 AM

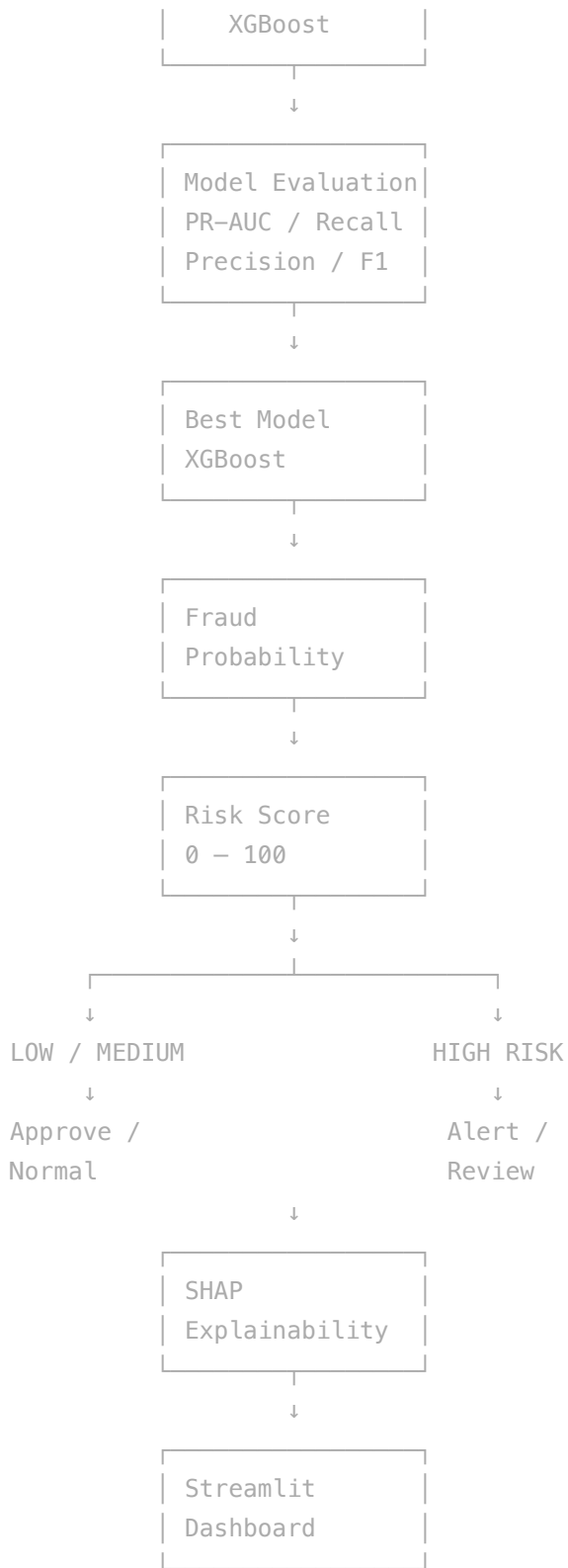
can you give me complete flow of project explained pointwise and also include which model used in

Absolutely. For your **Intelligent Fraud Detection & Risk Scoring System**, I recommend presenting the project as an **end-to-end ML pipeline**, not just "train a model and predict fraud."

## Intelligent Fraud Detection & Risk Scoring System

### 1. Overall project flow





## 2. Dataset

We use the **Credit Card Fraud Detection** dataset.

Your columns are:

Time  
V1

V2  
...  
V28  
Amount  
Class

## Important columns

V1–V28

These are anonymized numerical features.

We don't know their exact business meaning.

Amount

Transaction amount.

Class

Our target variable:

Class = 0 → Genuine  
Class = 1 → Fraud

So the ML problem is:

$$X = [Time, V1, V2, \dots, V28, Amount]$$

and

$$y = Class$$

## 3. Step 1 — Data collection

First we obtain the historical transaction dataset.

Historical Transactions  
↓  
creditcard.csv

Example:

| V1    | V2    | V3    | Amount | Class |
|-------|-------|-------|--------|-------|
| -1.35 | -0.07 | 2.53  | 149.62 | 0     |
| -2.31 | 1.95  | -1.60 | 0.00   | 1     |

The `Class` column gives us the known outcome for training.

## 4. Step 2 — Data understanding

Before training the model, we inspect:

- Number of rows
- Number of columns
- Data types
- Missing values
- Duplicate records
- Class distribution
- Feature distributions

For example:

```
df.shape
df.info()
df.isnull().sum()
df["Class"].value_counts()
```

### Why?

We need to understand the dataset before giving it to the ML algorithm.

## 5. Step 3 — Exploratory Data Analysis

We visualize the data.

### We investigate:

1. Fraud vs genuine transactions
2. Transaction amount distribution
3. Feature distributions
4. Correlations
5. Outliers
6. Class imbalance

For example:

```
Genuine transactions → ~99.8%
Fraud transactions   → ~0.2%
```

The exact numbers should be calculated from your copy of the dataset rather than assumed.

## 6. Step 4 — Handle class imbalance

This is one of the **most important parts of your project**.

Fraud transactions are much fewer than genuine transactions.

Suppose:

```
Genuine = 99,000  
Fraud    = 1,000
```

A model could simply predict everything as genuine and get:

```
Accuracy = 99%
```

But it would detect:

```
Fraud = 0
```

That's a terrible fraud detection system.

### Techniques we use

We can use:

#### Option 1 — Class weights

```
class_weight="balanced"
```

#### Option 2 — SMOTE

Synthetic Minority Oversampling Technique.

For your final project, I recommend experimenting with both and comparing the results.

## 7. Step 5 — Separate features and target

We separate:

X → Features

y → Target

**X**

Time

V1

V2

...

V28

Amount

**y**

Class

So:

```
X = df.drop("Class", axis=1)
y = df["Class"]
```

## 8. Step 6 — Train/Test split

We divide the dataset:

80% → Training

20% → Testing

Using:

```
train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

**Why stratify?**

Because fraud is rare.

It ensures that the training and testing sets maintain approximately the same fraud/genuine distribution.

## 9. Step 7 — Feature scaling

For models such as Logistic Regression, scaling is useful.

We use:

```
StandardScaler
```

Conceptually:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

This converts features to a comparable scale.

### Important

The scaler is fitted **only on training data**.

```
Training Data
  ↓
fit scaler
  ↓
transform training
  ↓
transform testing
```

We don't fit the scaler using test data because that can cause data leakage.

## 10. Step 8 — Model 1: Logistic Regression

Our first model is:

### Logistic Regression

It acts as a baseline.

It calculates a probability:

$$P(Fraud)$$



For example:

Transaction A → 0.02

Transaction B → 0.14

Transaction C → 0.87

So:

0.02 → Low likelihood

0.87 → High likelihood

## Why use it?

Because it is:

- Simple
- Fast
- Easy to explain
- Good baseline
- Provides probabilities

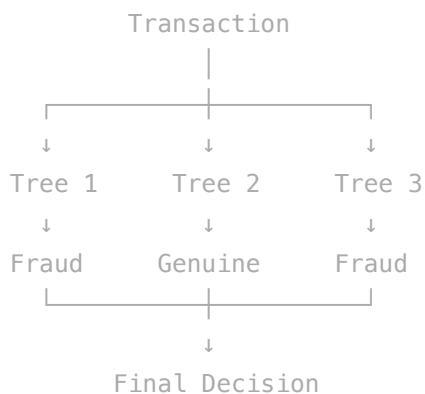
## 11. Step 9 — Model 2: Random Forest

Next we use:

### Random Forest Classifier

Random Forest consists of many decision trees.

Conceptually:



Each tree makes a prediction and the forest combines them.

### Why Random Forest?

It can capture nonlinear relationships and interactions between features.

## 12. Step 10 — Model 3: XGBoost

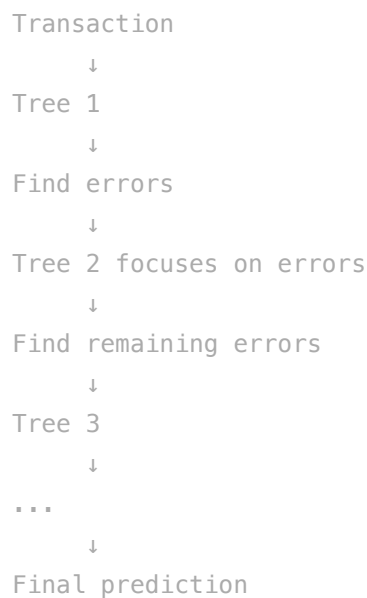
Our third and potentially strongest model is:

### XGBoost

XGBoost = **Extreme Gradient Boosting**.

It builds trees sequentially.

Conceptually:



### Why XGBoost?

It is particularly strong for structured/tabular data and can model complex nonlinear relationships.

For this project, **XGBoost will be our primary candidate for the final model**, but we should select the final model based on the actual validation/test results rather than assuming XGBoost will always win.

## 13. Model comparison

Now we have:



Random Forest



XGBoost

We compare them.

## Metrics

We should calculate:

- Precision
- Recall
- F1-score
- PR-AUC
- Confusion matrix

## 14. Why PR-AUC is particularly important

Because our dataset is highly imbalanced.

Precision-Recall AUC tells us how well the model identifies the minority class across different classification thresholds.

For example:

| Model               | PR-AUC |
|---------------------|--------|
| Logistic Regression | 0.75   |
| Random Forest       | 0.84   |
| XGBoost             | 0.88   |

These numbers are **examples only**; your actual values will come from training.

If XGBoost performs best, we select:

Final Model = XGBoost

## 15. Confusion matrix

We also visualize:

|                   | Predicted |       |
|-------------------|-----------|-------|
|                   | Genuine   | Fraud |
| Actual<br>Genuine | TN        | FP    |
| Fraud             | FN        | TP    |

The most concerning case is:

## FN — False Negative

Actual = Fraud  
Predicted = Genuine

This means:

The system missed a fraudulent transaction.

For fraud detection, reducing false negatives is often very important.

## 16. Threshold selection

This is an important improvement over simply using:

0.5

Suppose the model outputs:

Transaction A → 0.20  
Transaction B → 0.65  
Transaction C → 0.91

We could use:

Probability < 0.30 → LOW  
0.30–0.70 → MEDIUM  
≥ 0.70 → HIGH

But these thresholds should ideally be selected based on validation performance and business costs.

## 17. Fraud probability

The selected model gives:

```
model.predict_proba(transaction)
```

Example:

```
Genuine probability = 0.13  
Fraud probability   = 0.87
```

We use:

```
Fraud Probability = 87%
```

## 18. Risk score

Now we convert the probability into a score.

### Basic version

$$RiskScore = P(Fraud) \times 100$$

Example:

```
Fraud Probability = 0.87
```

Therefore:

```
Risk Score = 87
```

## 19. Risk classification

We can define:

```
0-29   → LOW  
30-69  → MEDIUM  
70-100 → HIGH
```

Example:

| Probability | Score | Risk |
|-------------|-------|------|
| 0.12        | 12    | LOW  |

| Probability | Score | Risk   |
|-------------|-------|--------|
| 0.45        | 45    | MEDIUM |
| 0.87        | 87    | HIGH   |

Again, these thresholds are configurable rather than universally correct.

## 20. Final decision

Now our system can produce:

```
Transaction
  ↓
ML Model
  ↓
Fraud Probability
  ↓
Risk Score
  ↓
Risk Level
  ↓
Decision
```

Example:

```
Transaction ID: TX1001

Fraud Probability: 91%
Risk Score: 91/100
Risk Level: HIGH

Action:
🚨 Send for additional verification
```

For a low-risk transaction:

```
Fraud Probability: 4%
Risk Score: 4/100
Risk Level: LOW

Action:
Normal processing
```

## 21. SHAP explainability

This is an excellent feature for your project.

A bank/user may ask:

**Why did the model give this transaction a high risk score?**

We use:

### SHAP — SHapley Additive exPlanations

It helps identify which features contributed most to a prediction.

Example:

```
Transaction
  ↓
XGBoost
  ↓
Risk = 91
  ↓
SHAP
  ↓
Important contributing features:
V14
V17
V10
V12
Amount
```

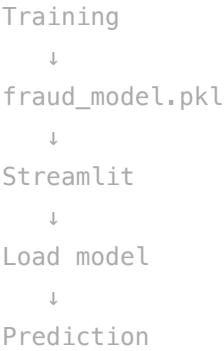
Remember that V1–V28 are anonymized, so we should **not invent real-world meanings for them.**

## 22. Save the model

After training:

```
models/
|
├─ fraud_model.pkl
├─ scaler.pkl
└─ features.pkl
```

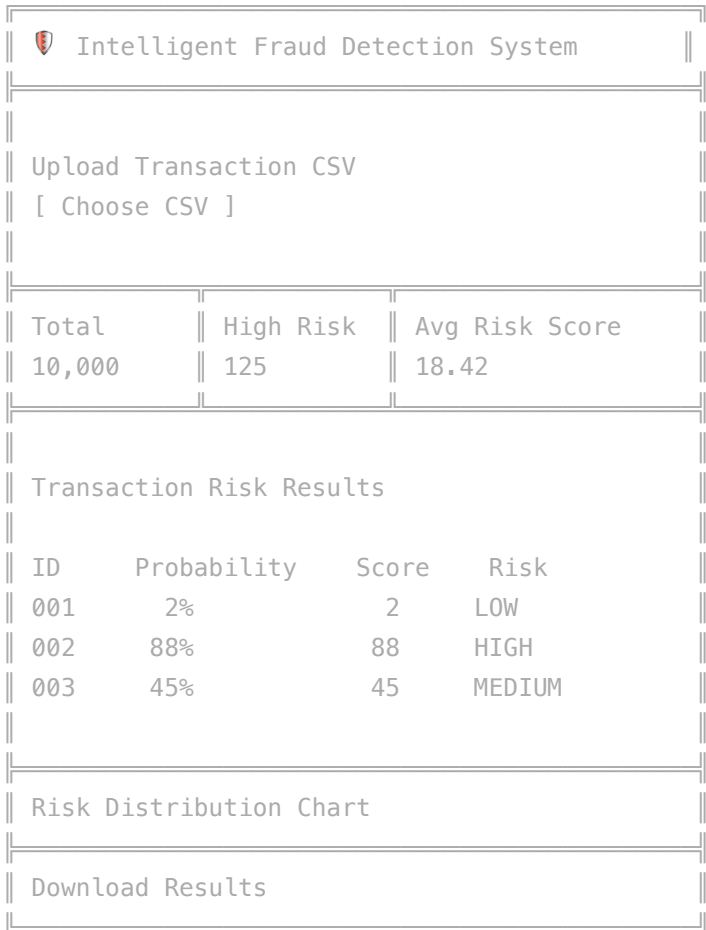
Then our application doesn't need to retrain the model every time.



## 23. Build Streamlit dashboard

Now we create the frontend.

### Dashboard



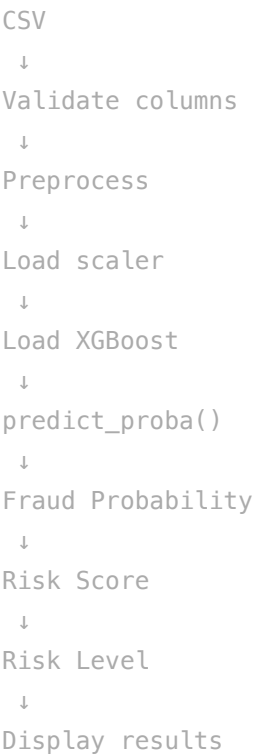
## 24. CSV prediction workflow

The user uploads:



transactions.csv

The application performs:



25. Example final prediction

Suppose the uploaded transaction produces:

Fraud probability = 0.923

The system displays:

|                                                                                                                         |            |
|-------------------------------------------------------------------------------------------------------------------------|------------|
| Transaction Risk Analysis                                                                                               |            |
| Fraud Probability: 92.3%                                                                                                |            |
| Risk Score:                                                                                                             | 92.3 / 100 |
| Risk Level:                                                                                                             | HIGH       |
|  Additional verification recommended |            |

## 26. Download results

The user can download:

```
fraud_predictions.csv
```

Example:

```
Transaction,Probability,Risk Score,Risk Level
TX001,0.02,2,LOW
TX002,0.87,87,HIGH
TX003,0.44,44,MEDIUM
```

## 27. Model monitoring

For a more advanced version, we can monitor:

```
Total transactions
Fraud rate
Average risk score
High-risk transactions
Medium-risk transactions
Low-risk transactions
```

Example:

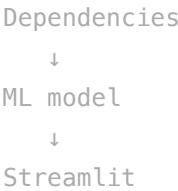
```
Today's Transactions      : 20,000
High Risk                 : 183
Medium Risk               : 1,021
Low Risk                  : 18,796
Average Risk Score        : 11.7
```

This makes the project look more like a real ML system rather than only a college notebook.

## 28. Docker

Finally, we containerize the application.

```
Docker
  ↓
Python environment
  ↓
```



This allows the application to run consistently on another machine.

## 29. Complete technology/model stack

Your final project will use:

| Layer                | Technology                        |
|----------------------|-----------------------------------|
| Programming          | Python                            |
| Data manipulation    | Pandas                            |
| Numerical processing | NumPy                             |
| Visualization        | Matplotlib + Seaborn              |
| Preprocessing        | Scikit-learn                      |
| Imbalance            | Class weights + SMOTE experiments |
| Model 1              | Logistic Regression               |
| Model 2              | Random Forest                     |
| Model 3              | <b>XGBoost</b>                    |
| Model selection      | PR-AUC + Recall + F1              |
| Explainability       | <b>SHAP</b>                       |
| Model serialization  | Joblib                            |
| Frontend             | <b>Streamlit</b>                  |
| Containerization     | <b>Docker</b>                     |
| Dataset              | Credit Card Fraud Detection       |

## 30. Project folder structure

I recommend we eventually make your project like this:

```
fraud-detection-system/  
|  
├─ data/  
|   ├── raw/  
|   │   └─ creditcard.csv  
|   └─ processed/  
|  
├─ notebooks/  
|   └─ fraud_analysis.ipynb  
|  
├─ src/  
|   ├── __init__.py  
|   ├── preprocessing.py  
|   ├── train.py  
|   ├── evaluate.py  
|   ├── predict.py  
|   └─ explain.py  
|  
├─ models/  
|   ├── fraud_model.pkl  
|   ├── scaler.pkl  
|   └─ features.pkl  
|  
├─ app/  
|   └─ app.py  
|  
├─ reports/  
|   ├── model_comparison.csv  
|   ├── confusion_matrix.png  
|   └─ shap_summary.png  
|  
├─ requirements.txt  
├─ Dockerfile  
├─ README.md  
└─ .gitignore
```

## 31. The complete project in 10 major phases

You can remember the project using these 10 phases:

### Phase 1 — Data

Collect Credit Card Fraud Dataset

## Phase 2 — EDA

Understand data  
Find imbalance  
Visualize patterns

## Phase 3 — Preprocessing

Clean data  
Separate X/y  
Scale features

## Phase 4 — Imbalance handling

Class weights  
SMOTE experiment

## Phase 5 — Model training

Logistic Regression  
Random Forest  
XGBoost

## Phase 6 — Model evaluation

Precision  
Recall  
F1  
PR-AUC  
Confusion Matrix

## Phase 7 — Model selection

Select best model  
↓  
XGBoost (if it wins validation)

## Phase 8 — Risk scoring

Fraud probability  
↓  
Risk Score 0–100  
↓  
LOW / MEDIUM / HIGH

## Phase 9 — Explainability

SHAP  
↓  
Why is this transaction risky?

## Phase 10 — Deployment

Streamlit  
↓  
Docker  
↓  
Deploy

# 32. How you should explain the project in an interview

If the interviewer asks:

### “Explain your project.”

You can say:

“I developed an end-to-end Intelligent Fraud Detection and Risk Scoring System using the Credit Card Fraud Detection dataset. Since the dataset is highly imbalanced, I used class-weighted learning and experimented with SMOTE. I trained and compared Logistic Regression, Random Forest, and XGBoost models. Instead of relying only on accuracy, I evaluated them using precision, recall, F1-score and PR-AUC. I selected the best-performing model and used its fraud probability to generate a 0–100 risk score and classify transactions into low, medium, and high-risk categories. I then integrated the model into a Streamlit dashboard and added SHAP explainability to understand the features contributing to individual predictions.”

That is a **much stronger explanation** than simply saying:

“I made a fraud detection model using Python.”

### 33. One important distinction

Your project actually has **two outputs**:

#### Output 1 — Classification

Fraud / Genuine

#### Output 2 — Risk scoring

0–100 Risk Score

For example:

Model probability = 0.91

Classification → FRAUD

Risk Score → 91/100

Risk Level → HIGH

So the project isn't merely a classifier. It is:

**Fraud Detection + Probability Estimation + Risk Scoring + Explainability +  
Interactive Dashboard**

That combination makes it much stronger as an **AI/ML portfolio project**.

### Recommended implementation order

Since you want the **complete working project**, I would build it in this exact order:

```
01 → Project setup
02 → Dataset loading
03 → EDA
04 → Preprocessing
05 → Class imbalance
06 → Logistic Regression
07 → Random Forest
08 → XGBoost
09 → Model comparison
10 → Select & save best model
11 → Risk scoring
12 → SHAP
13 → Streamlit
14 → CSV upload/prediction
15 → Dashboard
16 → Docker
```

[17 → README](#)[18 → Deployment](#)

The next practical step is **Phase 1–4: setting up the project and creating the complete preprocessing/EDA code**. After that, we can train the three models and build the final application without mixing notebook experimentation with production code.



ChatGPT can make mistakes. Check important info.